

BLM5106- Advanced Algorithm Analysis and Design

H. İrem Türkmen

[Introduction to the Design and Analysis of Algorithms](#), Anany Levitin

<http://ocw.mit.edu>, Design and Analysis of Algorithms

<http://web.stanford.edu/class/archive/cs/cs161/cs161.1176/>, Design and Analysis of Algorithms

<https://ceng.metu.edu.tr/>, Data Structures

Median Finding – Divide and Conquer

SELECT(S, i)

```

1  Pick  $x \in S$   $\triangleright$  cleverly
2  Compute  $k = \text{rank}(x)$ 
3   $B = \{y \in S \mid y < x\}$ 
4   $C = \{y \in S \mid y > x\}$ 
5  if  $k = i$ 
6      return  $x$ 
7  else if  $k > i$ 
8      return Select( $B, i$ )
9  else if  $k < i$ 
10     return Select( $C, i - k$ )
  
```

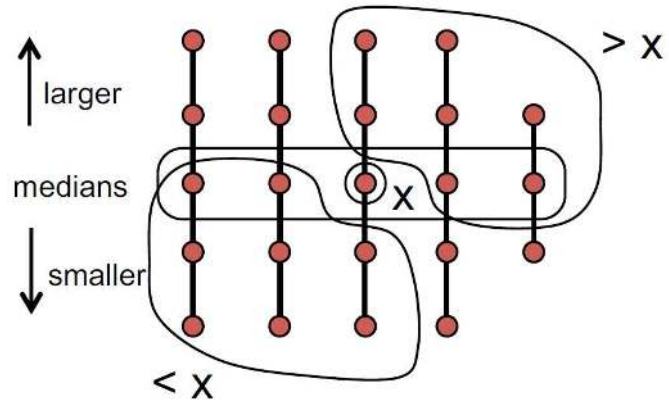
If len(S**) <= 50:**

- **S** = MergeSort(**S**)
- Return **S**[**i**]

i : rank that you want to find
K : rank of pivot

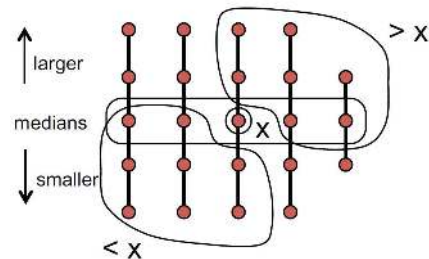
What about complexity?

- What are sizes of subgroups?



(bigger elements on top)

What about complexity?



$$3n/10 - 6$$

At least $3(\lceil \frac{n}{10} \rceil - 2)$ elements are $> x$

Recurrence:

$$T(n) = \begin{cases} O(1), & \text{for } n \leq 140 \\ T(\lceil \frac{n}{5} \rceil) + T(\frac{7n}{10} + 6) + \Theta(n), & \text{for } n > 140 \end{cases}$$

Median of medians

Discard one half (at most)

To partition columns
To sort $n/5$ columns
To divide array (smaller and bigger ones)

Some
reasonable
constant

Substitution by Mathematical Induction

- Master Teorem does not work in this case and recursion trees can get pretty messy here, since we have a recurrence relation that doesn't nicely break up our big problem into sub-problems of the same size.
- Instead, we will try to:
 - Make a guess
 - Check using an inductive argument
- This is called the substitution method : Assume your guess is true for $1, 2, \dots, n-1$, using this assumption you have to proof that your guess is also true for n

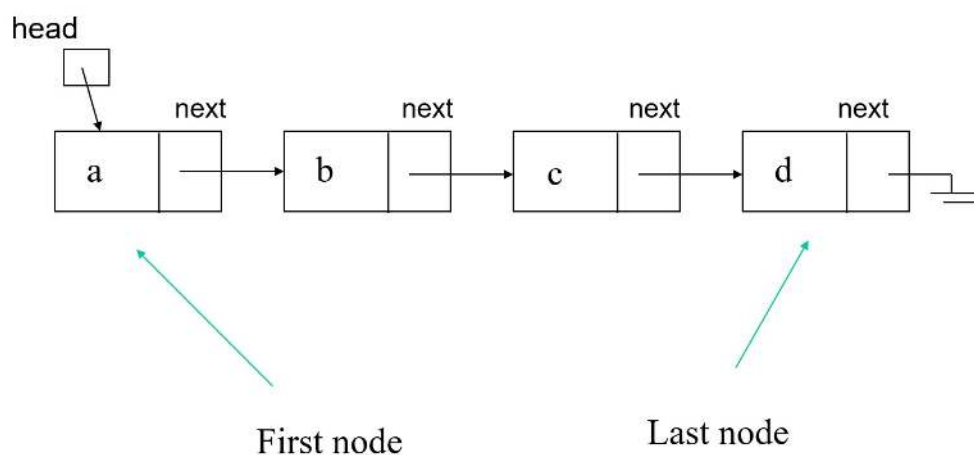
Substitution by Mathematical Induction

- Lets see some examples
- How to analyze divide and conquer median finding by mathematical induction?

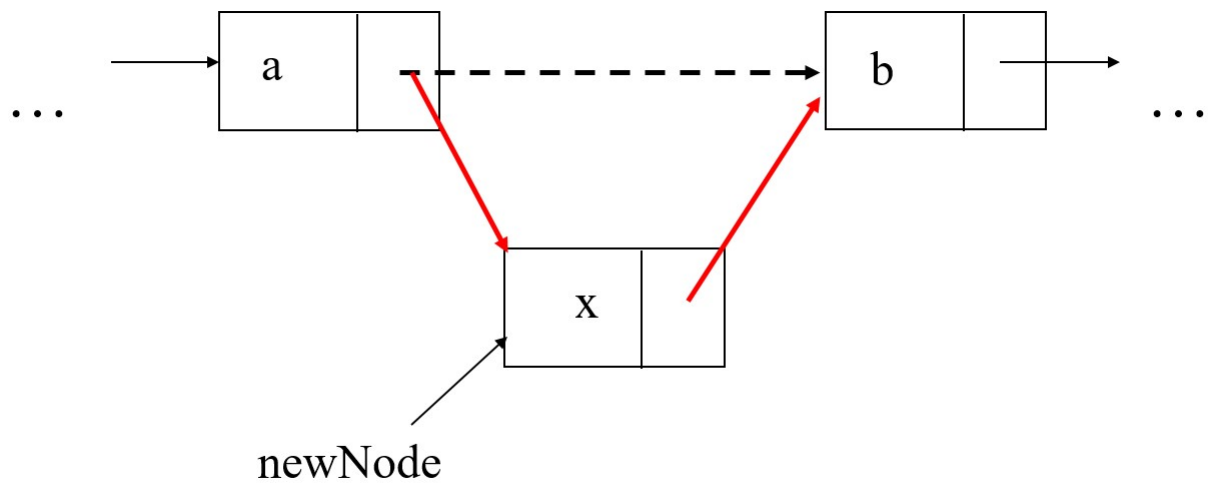
Basic Data Structures: Linked Lists

- Lets remember linked lists..
- Linked lists are used to store a collection of information (like arrays)
- A linked list is made of nodes that are pointing to each other
- We only know the address of the first node (head)
- Other nodes are reached by following the “next” pointers
- The last node points to NULL

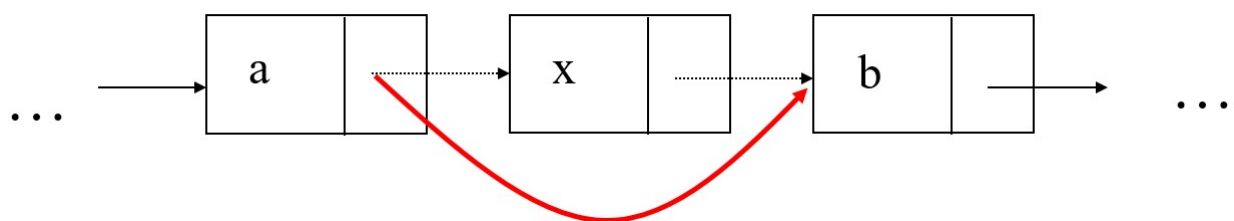
Linked Lists



Insert a node



Delete a node



```

- struct node{
    int val;
    struct node *next;
};

// listenin sonuna node ekler
- void push(struct node *head, int val) {
    struct node* current = head;
-    while (current->next != NULL) {
        current = current->next;
    }

    current->next = malloc(sizeof(struct node));
    current->next->val = val;
    current->next->next = NULL;
}

```

```

// listenin basina node ekler
- struct node* pushhead (struct node *head, int val) {
    struct node* newN;
    newN=malloc(sizeof(struct node));
    newN->next =head;
    newN->val =val;
    return newN;
}

// elemanları listeler
- void list(struct node *head)
{
    struct node* current = head;
    printf("liste elemanlari:\n");
-    while (current->next != NULL) {
        printf("-%d-",current->val);
        current = current->next;
    }
    printf("-%d-\n",current->val);
}

```

```

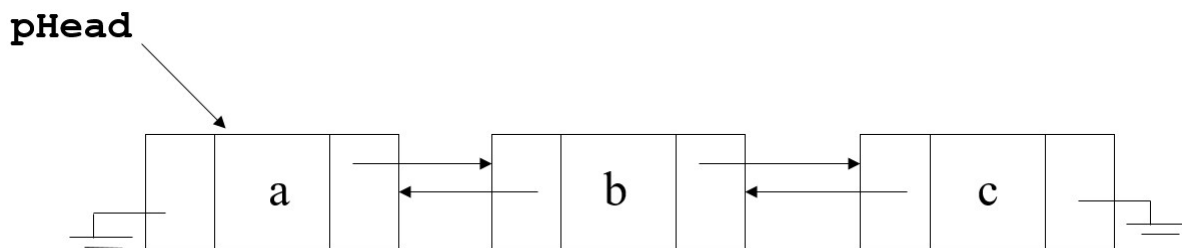
void deleteNode(struct node *head, int val)
{
    struct node* current = head, *before=NULL;

    while ((current->val != val)&&(current->next != NULL))
    {
        before=current;
        current = current->next;
    }

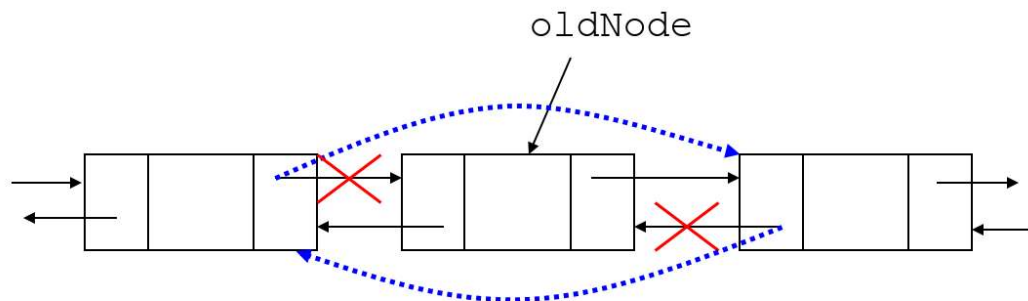
    if (current->val != val)
        printf("silinmek istenen eleman listede yok\n");
    else
    {
        before->next=current->next;
        free (current);
    }
}

```

Double Linked List

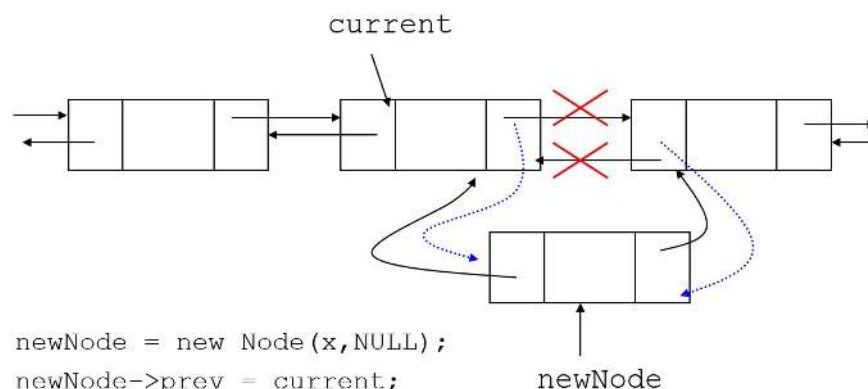


Delete a Node of Double Linked List



```
oldNode->prev->next = oldNode->next;
oldNode->next->prev = oldNode->prev;
delete oldNode;
```

Insert a Node to Double Linked List



```
newNode = new Node(x, NULL);
newNode->prev = current;
newNode->next = current->next;
newNode->prev->next = newNode;
newNode->next->prev = newNode;
```


A very efficient way to implement
dictionaries: **HASHING**

What is a dictionary?

- Dictionary is an abstract data type, a set with the operations of searching (lookup), insertion, and deletion defined on its elements.
- The elements of this set can be of an arbitrary nature: numbers, characters of some alphabet, character strings, and so on.
- Student records in a school, citizen records in a governmental office, book records in a library..

Dictionary

- Typically, records comprise several fields, each responsible for keeping a particular type of information about an entity the record represents.
- For example, a student record may contain fields for the student's ID, name, date of birth, sex, home address, major, and so on.
- Among record fields there is usually at least one called a **key** that is used for identifying entities represented by the records (e.g., the student's ID).
- We assume that we have to implement a dictionary of n records with keys $K_1, K_2, \dots, K_n$.

Hash Table and Hash Function

- **Hashing** is based on the idea of distributing keys among a one-dimensional array $H[0..m - 1]$ called a **hash table**.
- The distribution is done by computing, for each of the keys, the value of some predefined function h called the **hash function**.
- This function assigns an integer between 0 and $m - 1$, called the **hash address**, to a key.

Hash Table

- Many applications require a dynamic set that supports only the dictionary operations INSERT, SEARCH, and DELETE.
- For example, a compiler that translates a programming language maintains a symbol table, in which the keys of elements are arbitrary character strings corresponding to identifiers in the language.
- Although searching for an element in a hash table can take as long as searching for an element in a linked list ($O(n)$ time in the worst case), under reasonable assumptions, the average time to search for an element in a hash table is $O(1)$.

Hash Table

- When the number of keys actually stored is small relative to the total number of possible keys, hash tables become an effective alternative to directly addressing an array, since a hash table typically uses an array of size proportional to the number of keys actually stored.

Direct – address Tables

- Direct addressing is a simple technique that works well when the universe U of keys is reasonably small.
- Suppose that an application needs a dynamic set in which each element has a key drawn from the universe $U = \{0, 1, 2, \dots, m-1\}$, where m is not too large.
- We shall assume that no two elements have the same key.

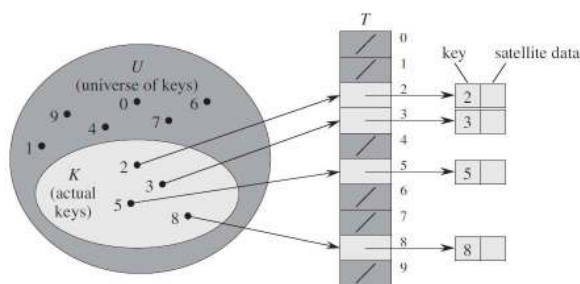


Figure 11.1 How to implement a dynamic set by a direct-address table T . Each key in the universe $U = \{0, 1, \dots, 9\}$ corresponds to an index in the table. The set $K = \{2, 3, 5, 8\}$ of actual keys determines the slots in the table that contain pointers to elements. The other slots, heavily shaded, contain NIL.

```
DIRECT-ADDRESS-SEARCH( $T, k$ )
1  return  $T[k]$ 
```

```
DIRECT-ADDRESS-INSERT( $T, x$ )
1   $T[x.key] = x$ 
```

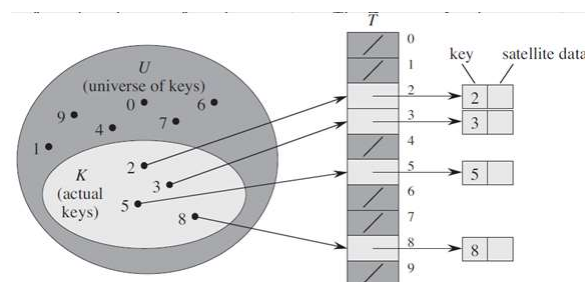
```
DIRECT-ADDRESS-DELETE( $T, x$ )
1   $T[x.key] = \text{NIL}$ 
```

Each of these operations takes only $O(1)$ time.

Dynamic Set with Direct Address Table

- Suppose that a dynamic set S is represented by a direct-address table T of length m . What is the worst-case performance of your procedure? Describe a procedure that finds the maximum element of S .

We start with the bottom of the table (the largest element) and scan the table backwards until we find a slot that contains an element. The worst case performance is $\Theta(m)$, if the maximum element is in the first position of the table (or the dynamic set is empty)



- The downside of direct addressing is obvious: if the universe U is large, storing a table T of size $|U|$ may be impractical, or even impossible, given the memory available on a typical computer.
- Furthermore, the set K of keys *actually stored* may be so small relative to U that most of the space allocated for T would be wasted.

Hashing

- With direct addressing, an element with key k is stored in slot k . With hashing, this element is stored in slot $h(k)$; that is, we use a **hash function** h to compute the slot from the key k .
- The hash function reduces the range of array indices and hence the size of the array.
- Instead of a size of $|U|$, the array can have size m .

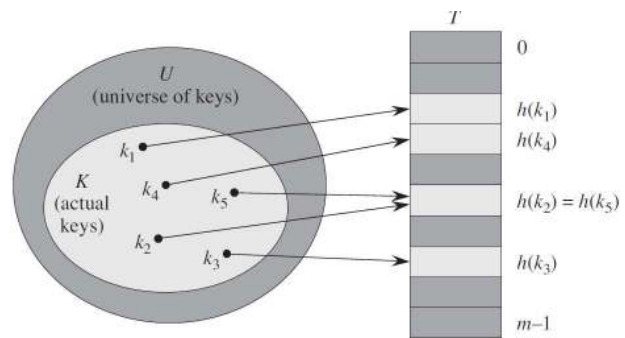


Figure 11.2 Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_5 map to the same slot, they collide.

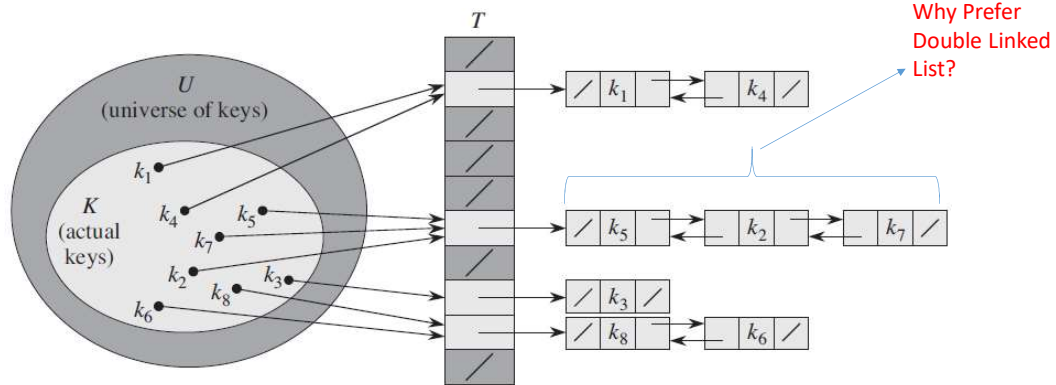
Collision

- There is one hitch: two keys may hash to the same slot. We call this situation a **collision**. Fortunately, we have effective techniques for resolving the conflict created by collisions.
- Of course, the ideal solution would be to avoid collisions altogether. We might try to achieve this goal by choosing a suitable hash function h .
- One idea is to make h appear to be “random,” thus avoiding collisions or at least minimizing their number.
- While a well designed, “random”-looking hash function can minimize the number of collisions, we still need a method for resolving the collisions that do occur.



Collision resolution by chaining

- In **chaining**, we place all the elements that hash to the same slot into the same linked list.



- Slot j contains a pointer to the head of the list of all stored elements that hash to j ; if there are no such elements, slot j contains NIL.

CHAINED-HASH-INSERT(T, x)

1 insert x at the head of list $T[h(x.key)]$

The worst-case running time for insertion is $O(1)$

CHAINED-HASH-SEARCH(T, k)

1 search for an element with key k in list $T[h(k)]$

The worstcase running time is proportional to the length of the list $O(n)$

CHAINED-HASH-DELETE(T, x)

1 delete x from the list $T[h(x.key)]$

The worst-case running time for insertion is $O(1)$

What if we use single linked list?

Analysis of hashing with chaining

- Lets see some examples